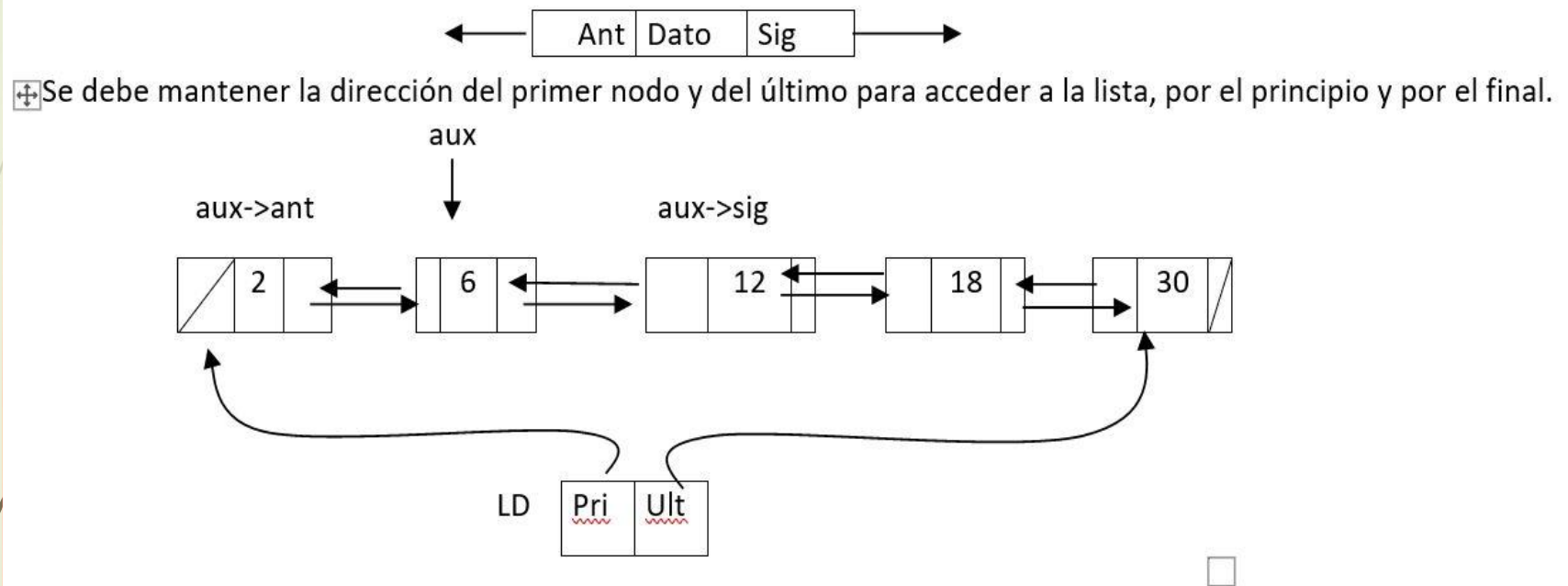


Listas Doblemente Enlazadas

En esta estructura cada nodo almacena la dirección del nodo siguiente y del nodo anterior. Esto permite recorrer la lista en ambos sentidos.



Listas DE - Ejercicio

Se tiene una lista simplemente enlazada que almacena números enteros sin repeticiones. Armar una lista doblemente enlazada ordenada de forma ascendente.

A partir de la lista generada, eliminar nodos de tal modo que en la misma no aparezcan dos valores pares o impares consecutivos.

Ejemplo: 10 7 13 8 12 5 → 5 7 8 10 12 13 → 5 8 13

Listas DE - Tipos

```
typedef struct nodo {  
    int num;  
    struct nodo * sig;} nodo;  
typedef struct nodo * TLista;
```

```
typedef struct nodoD {  
    int num;  
    struct nodoD *ant, * sig;} nodoD;  
typedef struct nodoD * PnodoD;
```

```
typedef struct {  
    PnodoD pri, ult;} TListaD;
```



```
void arma (TListaD *LD , TLista L){
    TLista aux;
    PnodoD nuevo, act;

    aux = L;
    (*LD).pri=NULL;
    (*LD).ult=NULL;
    while (aux != NULL) {
        nuevo = (PnodoD) malloc (sizeof(nodoD));
        nuevo->num = aux->num;
        if ((*LD).pri == NULL || nuevo->num < (*LD).pri->num) {
            nuevo->sig = (*LD).pri;
            nuevo->ant = NULL;
            if ((*LD).pri == NULL)
                (*LD).ult = nuevo;
            else
                (*LD).pri->ant = nuevo;
            (*LD).pri = nuevo;
        }
        else
            if (nuevo->num > (*LD).ult->num) {
                nuevo->ant = (*LD).ult;
                nuevo->sig = NULL;
                (*LD).ult->sig = nuevo;
                (*LD).ult = nuevo;
            }
            else {
                act = (*LD).pri;
                while (nuevo->num > act->num)
                    act= act->sig;
                nuevo->sig = act;
                nuevo->ant = act->ant;
                act->ant->sig = nuevo;
                act->ant = nuevo;
            }
        aux = aux->sig;
    }
}
```



```
void eliminaVarios (TListaD * LD) {
    PnodoD elim, act;

    if ((*LD).pri != NULL) {
        act = (*LD).pri->sig;
        while (act != NULL)
            if (act->num % 2 == act->ant->num % 2) {
                elim = act;
                act = act->sig;
                if (elim->sig == NULL) {
                    (*LD).ult = elim->ant;
                    (*LD).ult-> sig = NULL;
                }
                else {
                    elim->ant->sig= elim->sig;
                    elim->sig->ant= elim->ant;
                }
                free (elim);
            }
        else
            act = act->sig;
    }
}
```